



Desenvolvimento Web

Profª Sabrina B. Moreira



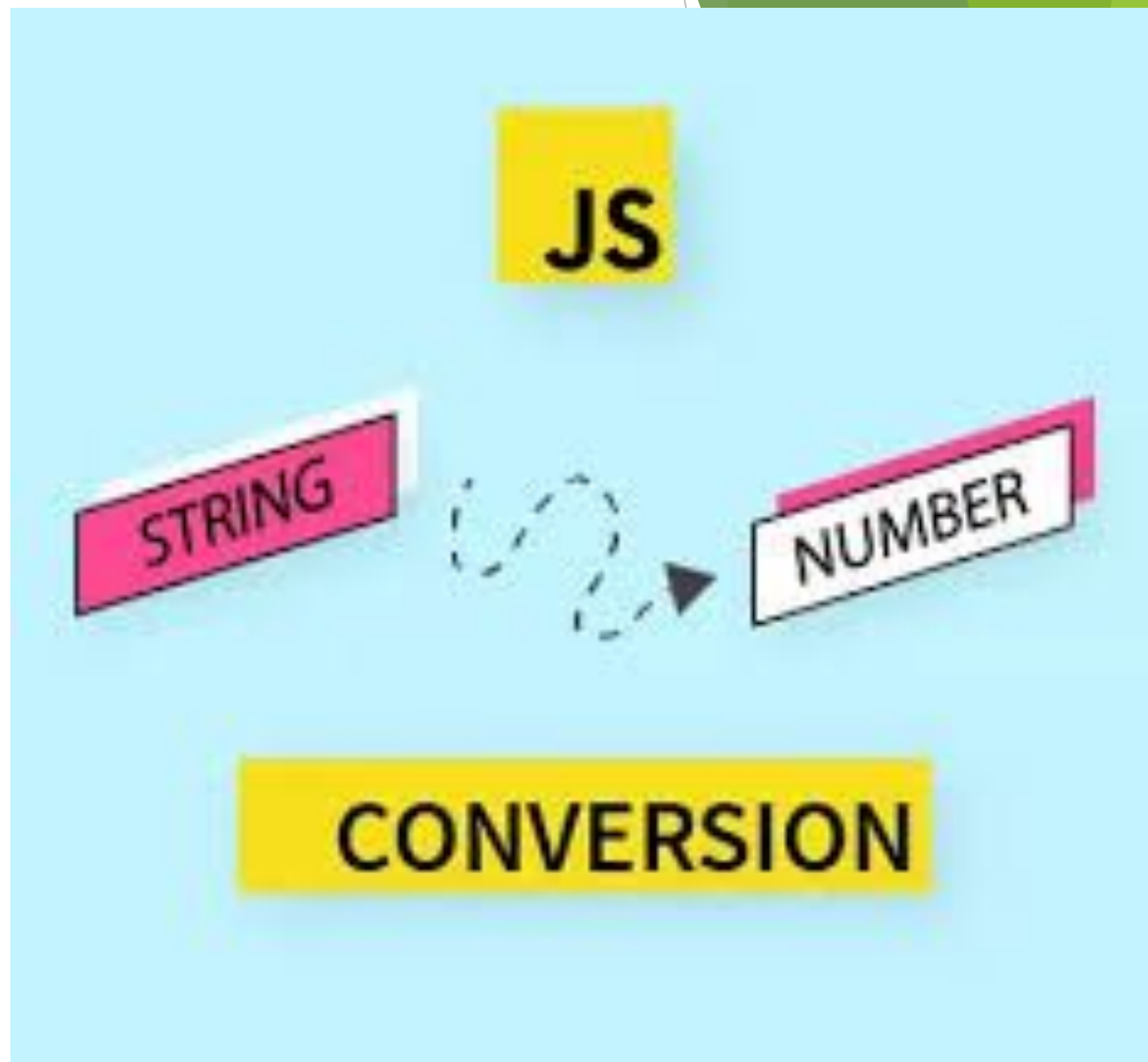
JavaScript

Aula 17.1: Conversão e verificação de tipos, métodos auxiliares, função e estruturas condicionais

Conversão de tipos

JS: conversão de tipos

- ❖ Em JavaScript, é comum receber dados em um formato e precisar utilizá-los em outro.
- ❖ Por exemplo, valores digitados em formulários HTML geralmente são recebidos como texto (*string*), mesmo quando representam números.



JS: conversão de tipos

- ❖ A conversão de tipos (*type conversion*) é o processo de transformar um valor de um tipo em outro.
- ❖ Permitindo que ele seja utilizado corretamente em cálculos, comparações e outras operações.



Fonte: Maldonado, 2021

Coerção de tipos

JS: Coerção de tipos

- ❖ Além das conversões realizadas manualmente pelo desenvolvedor, o JavaScript também pode converter valores automaticamente durante **determinadas operações**.
- ❖ Esse comportamento é conhecido como coerção de tipos (*type coercion*) e ocorre quando a linguagem tenta **adaptar os operandos** envolvidos em uma expressão.

JS: Coerção de tipos

- ❖ A **coerção de tipos** ocorre porque o JavaScript é uma linguagem de tipagem dinâmica, ou seja, as variáveis podem armazenar **diferentes tipos** de valores ao longo da execução do programa sem a necessidade de uma **declaração explícita**.
- ❖ Essa característica oferece maior flexibilidade e agilidade no desenvolvimento, permitindo escrever código com **menos restrições**.

```
> let dynamicTyping = 10;
    typeof dynamicTyping;

< 'number'

> dynamicTyping = "dez";
    typeof dynamicTyping;

< 'string'

> dynamicTyping = "10";
    typeof dynamicTyping;

< 'string'

>
    dynamicTyping = true;
    typeof dynamicTyping;

< 'boolean'
```


JS: Coerção de tipos

- ❖ Isso pode ser útil em alguns casos, mas também é perigoso porque pode gerar resultados inesperados e erros difíceis de identificar.
- ❖ Por exemplo, quando valores de tipos diferentes são utilizados em operações matemáticas, comparações ou concatenações sem a devida validação.

```
let firstValue = "5";  
let secondValue = 1;  
firstValue + secondValue;  
'51'  
firstValue - secondValue;  
4
```

Fonte: autora. 2026

Verificação de tipos

JS: verificação de tipo

- ❖ Antes de realizar operações matemáticas com dados recebidos do usuário, uma boa prática é verificar os tipos dos dados, e se for necessário, os converter manualmente os valores para o tipo adequado.
- ❖ E com isso, garantindo que os tipos estejam corretos e o resultado seja previsível.

```
> typeof firstValue;  
< 'string'  
> typeof secondValue;  
< 'number'  
> Number(firstValue) + secondValue;  
< 6  
> firstValue - secondValue;  
< 4  
> Number(firstValue) - secondValue;  
< 4
```

JS: verificação de valores nulos

- ❖ Ao verificarmos o tipo de um *null*, é retornado o valor “*object*”, o que parece incorreto já que *null* não é um objeto.
- ❖ Isso acontece devido a uma falha histórica presente desde as primeiras versões do JavaScript.
- ❖ Por questões de **compatibilidade** com milhões de aplicações existentes, esse comportamento **nunca** foi corrigido.

```
> typeof null;  
< 'object'  
  
> typeof undefined;  
< 'undefined'
```

Fonte: autora. 2026

JS: verificação de valores nulos

- ❖ Por esse motivo, não é recomendado utilizar typeof para verificar se um valor é nulo.
- ❖ A forma correta é utilizar a comparação estrita (===).
- ❖ Também não se deve verificar com === "*object*", pois objetos reais também retornam "object" e o resultado pode gerar interpretações incorretas.

```
> let nullValue = null;
  typeof nullValue;
< 'object'

> typeof null === "object";
< true

> nullValue === null;
< true
```

Conversão manual de tipos

Conversão para Number

JS: conversão para tipo Number

- ❖ Para utilizar valores em **cálculos matemáticos**, é necessário convertê-los para o tipo *number*.
- ❖ O JavaScript disponibiliza funções que permitem **converter textos em números**, como *Number()*, *parseInt()* e *parseFloat()*.
- ❖ Cada uma delas possui comportamentos específicos e deve ser utilizada de acordo com a necessidade da aplicação.

JS: conversão para tipo Number

Função	O que faz	Exemplo	Resultado
Number()	Converte o valor inteiro para um número	Number("10")	10
parseInt()	Extraí a parte inteira de uma string	parseInt("10.9")	10
parseFloat()	Extraí um número decimal de uma string	parseFloat("10.9")	10.9

JS: conversão para tipo Number

- ❖ Cada um dos tipos de conversão tem suas características: *Number()* tenta converter **todo o valor** para um número.
- ❖ *parseInt()* extrai apenas a **parte inteira**.
- ❖ *parseFloat()* permite converter números com **casas decimais**.

```
// Número
Number("10.99"); // 10.99
Number("10.99 anos"); // NaN

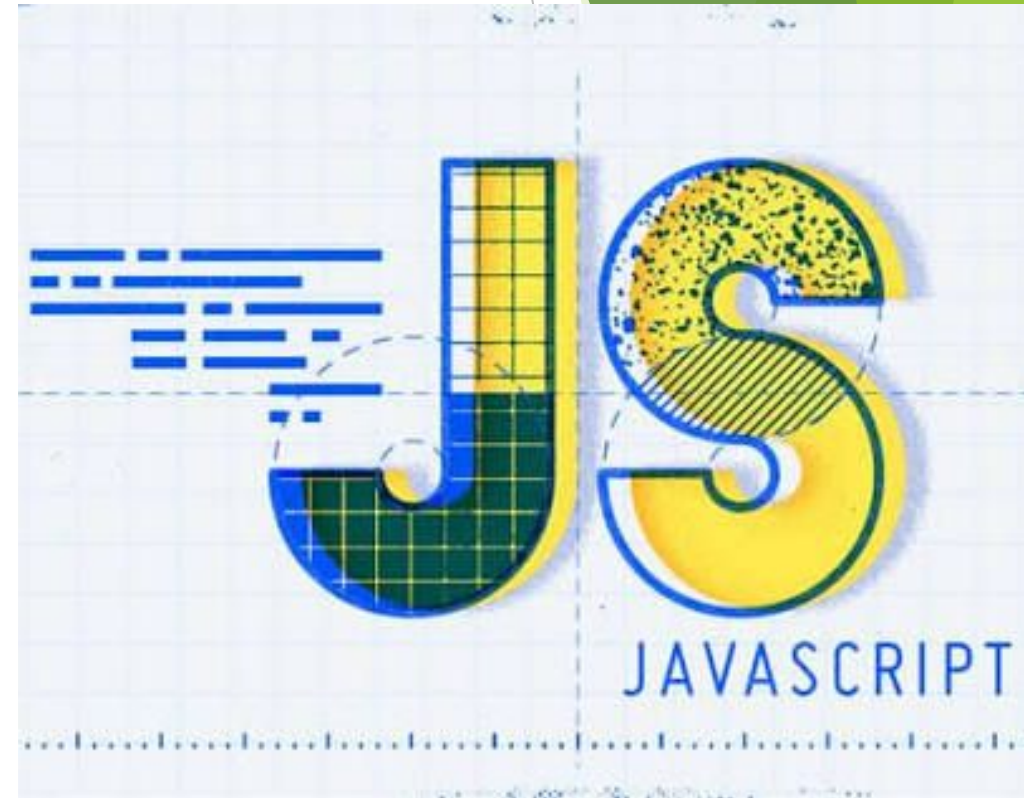
// Número inteiro
parseInt("10.99"); // 10
parseInt("10.99 anos"); // 10
parseInt("há 10.99 anos"); // NaN

// Número de ponto flutuante
parseFloat("10.99"); // 10.99
parseFloat("10.99 anos"); // 10.99
parseFloat("anos 10.99"); // NaN
```

Conversão para String

JS: conversão para tipo String

- ❖ Para trabalhar com valores como texto, é necessário convertê-los para o tipo **string**.
- ❖ O JavaScript disponibiliza recursos que permitem transformar números, valores booleanos e outros tipos em **texto**, como *String()* e o método *toString()*.
- ❖ Cada abordagem deve ser utilizada conforme o contexto da aplicação.



Fonte: Tagliaferri, 2021

JS: conversão para tipo String

Função	O que faz	Exemplo	Resultado
String()	Converte qualquer valor para texto, incluindo null e undefined	String(null)	"null"
toString()	Converte qualquer valor para texto (exceto null e undefined), ou seja, quando tem certeza da existência de valor.	(123).toString()	"123"

JS: String()

- ❖ A função *String()* é utilizada para **converter valores** de qualquer tipo para texto (*string*).
- ❖ Pode converter números, booleanos, datas, objetos e até mesmo os valores especiais null e undefined sem gerar erros.
- ❖ Realiza uma **conversão explícita e segura**, é uma das formas mais recomendadas para transformar dados em texto.

```
String(10);  
'10'  
  
String(10.99);  
'10.99'  
  
String(true);  
'true'  
  
String(null);  
'null'  
  
String(undefined);  
'undefined'  
  
String({name: "oi", id: 1});  
'[object Object]'
```


JS: toString()

- ❖ Já o método *toString()* também converte valores para texto.
- ❖ Porém só pode ser utilizado em dados que possuem um valor válido.

```
> (10).toString();  
< '10'  
  
> (10.99).toString();  
< '10.99'  
  
> (true).toString();  
< 'true'
```

Fonte: autora, 2026

JS: toString()

- ❖ Diferentemente de *String()*, ele **não funciona** com *null* e *undefined*, gerando um erro caso seja chamado nesses valores.
- ❖ É bastante utilizado para converter números, booleanos e objetos para sua representação textual.

```
> (null).toString();
✖ ▶ Uncaught TypeError: Cannot read properties of null (reading 'toString')
   at <anonymous>:1:8

> (undefined).toString();
✖ ▶ Uncaught TypeError: Cannot read properties of undefined (reading 'toString')
   at <anonymous>:1:13

> ({name: "oi", id: 1}).toString();
< '[object Object]'
```


JS: diferença entre `String()` e `toString()`

- ❖ Desta forma, apesar de ambos converterem **valores para texto** a função `String()` consegue converter qualquer valor (incluindo *null* e *undefined*), enquanto `toString()` não pode ser utilizado em valores *null* ou *undefined*, pois eles não possuem esse método.
- ❖ Em situações em que o valor pode ser **desconhecido** ou vir de fontes externas, como formulários e APIs, `String()` costuma ser a **opção mais segura**.

Conversão para Boolean

JS: conversão para tipo Boolean

- ❖ O tipo boolean representa apenas dois valores: *true* ou *false*.
- ❖ Ele é amplamente utilizado em estruturas de decisão, validações e controle de fluxo da aplicação.
- ❖ Em JavaScript, praticamente qualquer valor pode ser convertido para booleano.

```
let tentativas = 0;
const senhaCorreta = "123";
let senhaInformada = "";

while (tentativas < 3) {
  // Controle de fluxo
  senhaInformada = prompt("Digite a senha:");

  const senhaValida = senhaInformada === senhaCorreta;

  if (senhaValida) {
    // Estrutura de decisão
    console.log("Acesso permitido.");
    alert("Acesso permitido.");
    break;
  } else {
    console.log("Senha incorreta.");
  }

  tentativas++;
}

Acesso permitido.
```

JS: truthy values

- ❖ As conversões explícitas são realizadas pela função *Boolean()* e a conversão implícita utilizada em estruturas condicionais.
- ❖ Em geral, a maioria dos valores é convertida para *true*, incluindo números diferentes de zero, strings que contenham qualquer caractere (até mesmo um espaço em branco), objetos e arrays (com valor ou não).

```
> Boolean(1);  
◀ true  
  
> Boolean(-1);  
◀ true  
  
> Boolean("JavaScript");  
◀ true  
  
> Boolean(" ");  
◀ true  
  
> Boolean({});  
◀ true  
  
> Boolean([]);  
◀ true
```

JS: falsy values

Apenas alguns valores específicos são convertidos para false:

- ❖ false, 0, -0, 0n (BigInt zero)
- ❖ "" (string vazia)
- ❖ null, undefined e NaN.

Esses valores são conhecidos como *falsy values*. Todos os demais são considerados *truthy values* e são convertidos para *true* quando utilizados em estruturas de decisão e operadores lógicos.

```
> Boolean(false);  
< false  
> Boolean(0);  
< false  
> Boolean(-0);  
< false  
> Boolean(0n);  
< false  
> Boolean("");  
< false  
> Boolean(null);  
< false  
> Boolean(undefined);  
< false  
> Boolean(NaN);  
< false
```

JS: conversão de booleanos para outros tipos

- ❖ Valores booleanos também podem ser convertidos para **outros tipos de dados** em JavaScript.
- ❖ Quando convertidos para números, true torna-se 1 e false torna-se 0.
- ❖ Ao serem convertidos para strings, passam a ser representados pelos **textos** "true" e "false".
- ❖ Pode ocorrer de forma explícita, utilizando funções como *Number()* e *toString()*, ou implicitamente em **determinadas operações** realizadas pela linguagem.

```
> Number(true);  
< 1  
> Number(false);  
< 0  
> true.toString();  
< 'true'  
> false.toString();  
< 'false'  
> true + 1;  
< 2  
> false + 1;  
< 1  
> true * 10;  
< 10  
> false / 10;  
< 0
```

Métodos auxiliares

JS: métodos auxiliares

- ❖ Os tipos de dados do JavaScript disponibilizam diversos métodos auxiliares, que são **funções prontas** criadas para facilitar tarefas comuns do desenvolvimento.
- ❖ Esses métodos permitem manipular textos, realizar cálculos, percorrer arrays, formatar datas, validar informações e executar diversas operações sem a necessidade de criar algoritmos do zero.
- ❖ Aumenta a produtividade, torna o **código mais legível** e ajuda a aproveitar melhor os **recursos oferecidos** pela própria linguagem.

Métodos para numbers

JS: métodos para numbers

Método	Descrição	Exemplo	Resultado
toFixed()	Define a quantidade de casas decimais e retorna uma string	(10.567).toFixed(2);	"10.57"
toString()	Converte o número para string	(10).toString();	"10"
Number.isNaN()	Verifica se o valor é exatamente NaN	Number.isNaN(NaN);	true
Number.isInteger()	Verifica se o número é inteiro	Number.isInteger(10)	true
parseInt()	Converte um valor para inteiro	parseInt("10.9")	10
parseFloat()	Converte um valor para decimal	parseFloat("10.9")	10.9

Métodos para Strings

JS: métodos para Strings

Método	Descrição	Exemplo	Resultado
length	Retorna a quantidade de caracteres da string	"JavaScript".length	10
toUpperCase()	Converte todos os caracteres para maiúsculo	"JavaScript".toUpperCase()	"JAVASCRIPT"
toLowerCase()	Converte todos os caracteres para minúsculo	"JavaScript".toLowerCase()	"javascript"
trim()	Remove espaços do início e do fim da string	" JavaScript ".trim()	"JavaScript"
includes()	Verifica se um texto está contido na string	"JavaScript".includes("Script")	true
startsWith()	Verifica se a string começa com determinado texto.	"JavaScript".startsWith("Java")	true

JS: métodos para Strings

Método	Descrição	Exemplo	Resultado
endsWith()	Verifica se a string termina com determinado texto	"JavaScript".endsWith("Script");	true
replace()	Substitui a primeira ocorrência encontrada	"Ana Ana".replace("Java", "Type");	"Ana Maria"
replaceAll()	Substitui todas as ocorrências encontradas	"Ana Ana".replaceAll("Ana", "Maria");	"Maria Maria"
split()	Divide uma string em um array pelo separador indicado	"HTML,CSS,JS".split(",");	["HTML", "CSS", "JS"]
substring()	Extraí parte da string utilizando posição inicial e final	"JavaScript".substring(0,4);	"Java"
slice()	Extraí parte da string. Aceita índices negativos	"JavaScript".slice(-6);	"Script"
charAt()	Retorna o caractere de uma posição específica	"JavaScript".charAt(4);	"S"

Métodos para booleanos

JS: métodos para booleanos

Método	Descrição	Exemplo	Resultado
toString()	Converte o booleano para texto	true.toString();	"true"
valueOf()	Retorna o valor primitivo do booleano	true.valueOf()	true

Fonte: autora, 2026

JS: Template strings (template literals)

- ❖ Foram introduzidas no *ECMAScript 2015* para facilitar a criação e manipulação de textos. Utilizam o acento grave (`` `` ``) em vez de aspas simples ou duplas e permitem inserir variáveis e expressões diretamente dentro da string por meio da sintaxe ``${}``. Suportam textos em múltiplas linhas sem a necessidade de caracteres especiais para quebra de linha.

```
let languageName = "Javascript";
console.log("Estamos aprendendo " + languageName + "!");

Estamos aprendendo Javascript!

console.log(`Estamos aprendendo ${languageName}!`);

Estamos aprendendo Javascript!
```

Função

JS: funções

- ❖ As funções são **blocos de código** reutilizáveis criados para executar uma tarefa específica.
- ❖ Elas permitem organizar melhor o programa, evitar a repetição de código e facilitar a manutenção da aplicação.
- ❖ Uma função pode receber informações por meio de parâmetros, processar esses dados e retornar um resultado para quem a chamou.

```
function saudar() {  
  console.log("Olá!");  
}  
  
saudar();  
  
Olá!
```

Fonte: autora, 2026

JS: funções

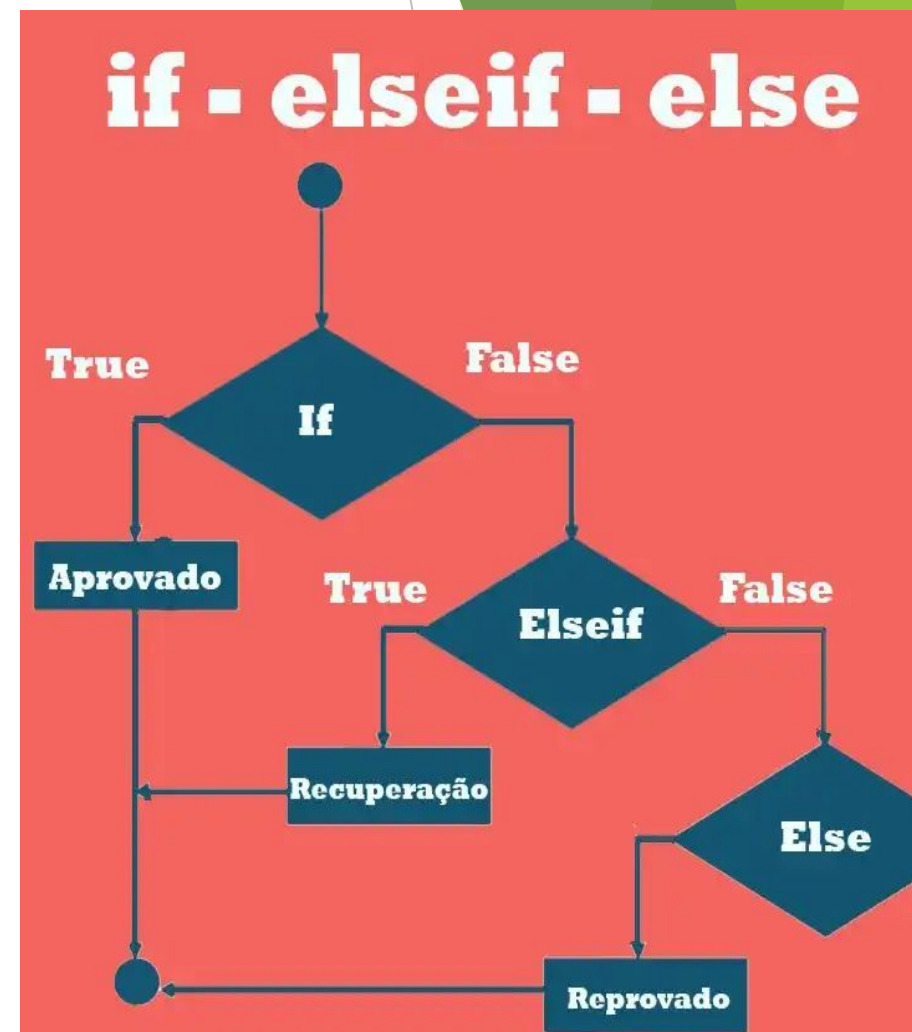
- ❖ As funções são **amplamente utilizadas** para implementar regras de negócio, realizar cálculos, manipular dados, responder a eventos e dividir aplicações complexas em partes menores e mais fáceis de compreender. Além disso, podem ser armazenadas em variáveis, passadas como parâmetros e retornadas por outras funções.

```
function sum(number1, number2) {  
  return Number(number1) + Number(number2);  
}  
  
sum(1,1);  
2  
  
sum(2,3);  
5
```

Estruturas condicionais

JS: Estruturas condicionais

- ❖ As estruturas condicionais permitem que um programa **tome decisões** durante sua execução.
- ❖ Elas avaliam uma condição que resulta em um valor booleano (true ou false) e determinam qual bloco de código deve ser executado.
- ❖ É fundamental para criar aplicações que reagem a **diferentes situações**, como validar dados e executar ações específicas de acordo com o contexto.



JS: If

- ❖ É utilizada para criar um desvio no nosso código.
- ❖ Se a **condição avaliada** for verdadeira (*true*), o bloco do *if* será executado, caso contrário, seguirá o restante da execução.

```
> let mediaDeNotas = 7;  
   const mediaParaAprovacao = 6;  
  
   if (mediaDeNotas >= mediaParaAprovacao) {  
       console.log("Aprovado");  
   }  
  
Aprovado
```


JS: Else

- ❖ Quando temos mais de um desvio de **caminho** para nosso código, podemos trabalhar com o *else* em conjunto ao *if*.
- ❖ Se a condição avaliada for **verdadeira** (*true*), o bloco do *if* será executado, caso contrário, o bloco do *else* será executado.

```
> let mediaDeNotas = 7;  
   const mediaParaAprovacao = 6;  
  
   if (mediaDeNotas >= mediaParaAprovacao) {  
     console.log("Aprovado");  
   } else {  
     console.log("Reprovado");  
   }
```

Aprovado

JS: Else If

- ❖ Permite avaliar múltiplas condições em sequência, é utilizado quando há várias possibilidades de decisão.
- ❖ Quando a condição do *if* é falsa, o programa verifica as condições dos blocos *else if* até encontrar uma que seja verdadeira. Caso nenhuma condição seja, o bloco *else if*, se existir, será executado.

```
let mediaDeNotas = 7;
const mediaParaAprovacao = 6;
const mediaParaMerito = 9;

if (mediaDeNotas >= mediaParaMerito) {
  console.log("Aprovação com mérito");
} else if (mediaDeNotas >= mediaParaAprovacao) {
  console.log("Aprovado");
} else {
  console.log("Reprovado");
}
```

Aprovado

Fonte: autora, 2026

JS: Operador Ternário

- ❖ O operador ternário é uma forma simplificada de **escrever estruturas *if* / *else*** quando a decisão envolve apenas duas possibilidades.
- ❖ Ele utiliza a sintaxe condição ? *valorSeVerdadeiro* : *valorSeFalso*, permitindo **atribuir valores** ou **executar expressões** de maneira mais compacta. Apesar de **conciso**, seu uso excessivo pode **dificultar a leitura** do código.

```
> let mediaDeNotas = 7;  
  const mediaParaAprovacao = 6;  
  if  
  mediaDeNotas >= mediaParaAprovacao ? console.log("Aprovado") : console.log("Reprovado");  
  else  
  Aprovado
```

JS: Switch

- ❖ Se usa quando uma variável ou expressão pode assumir vários valores específicos conhecidos.
- ❖ Em vez de criar diversos blocos *if / else if*, o switch permite organizar as comparações por meio de casos (case).
- ❖ Quando um dos casos corresponde ao valor avaliado, o bloco associado é executado.

```
let situacao = "Aprovado";  
switch (situacao) {  
  case "Aprovação com mérito":  
    console.log("Parabéns pelo excelente desempenho!");  
    break;  
  
  case "Aprovado":  
    console.log("Aluno aprovado.");  
    break;  
  
  case "Reprovado":  
    console.log("Aluno reprovado.");  
    break;  
  
  default:  
    console.log("Situação inválida.");  
}  
  
Aluno aprovado.
```



JS: Operador OR (||)

- ❖ É utilizado para verificar **múltiplas condições** ou fornecer valores padrão.
- ❖ Em expressões, ele retorna o primeiro valor considerado verdadeiro (truthy) que encontrar. Caso todos os valores sejam considerados falsos (*falsy*), ele retorna o último valor avaliado.

```
let userName = "";  
console.log(userName || "Visitante");  
  
Visitante  
  
userName = "Sabrina";  
console.log(userName || "Visitante");  
  
Sabrina
```

Fonte: autora, 2026

JS: Operador Nullish Coalescing (??)

- ❖ Esse operador só considera a substituição quando o **valor à esquerda** for exatamente *null* ou *undefined*. Valores como *0*, *false* e *""* (string vazia) são preservados.
- ❖ Isso torna o operador ?? especialmente útil quando esses valores possuem **significado válido** na aplicação e não devem ser substituídos por um valor padrão.

```
console.log(userName ?? "Visitante");  
Sabrina  
  
userName = null;  
console.log(userName ?? "Visitante")  
Visitante
```


The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Dúvidas?

Vídeos complementares

- Assista os vídeos abaixo para revisar o conteúdo dessa aula.



Exercício

Exercício 1

- ❖ Vamos montar um script que verifica se um **acadêmico foi aprovado** na disciplina.
- ❖ Primeiro, devemos receber a média final do acadêmico e verificar se ela é **igual ou maior que 6**.
- ❖ Depois, devemos receber o total de faltas e calcular sua frequência na disciplina. Se a **frequência for menor que 75%**, o acadêmico será reprovado.
- ❖ A **aprovação final** deverá ser definida pela aprovação por nota E por frequência.

Exercício 1

❖ Primeira função:

```
> function verificacaoAprovacaoPorNotas() {  
  const mediaParaAprovacao = 6;  
  const mediaDeNotas = Number(  
    prompt("Digite a média acadêmica do semestre: "),  
  );  
  if (mediaDeNotas >= mediaParaAprovacao) {  
    // media das notas do acadêmico deve ser maior ou igual a média para aprovação  
    return true;  
  } else {  
    console.log("Reprovado por média: ", mediaDeNotas);  
    return false;  
  }  
}
```

```
verificacaoAprovacaoPorNotas();
```

```
Reprovado por média: 5
```

Exercício 1

❖ Segunda função:

```
function verificacaoAprovacaoPorFrequencia() {
  const porcentagemFrequenciaMinimaParaAprovacao = 75;
  const totalAulas = 72;
  const totalDeFaltas = Number(
    prompt("Digite o total de faltas do acadêmico no semestre: "),
  );
  const porcentagemFrequenciaTotal =
    ((totalAulas - totalDeFaltas) * 100) / totalAulas; // porcentagem da frequência do acadêmico

  if (
    porcentagemFrequenciaTotal >= porcentagemFrequenciaMinimaParaAprovacao
  ) {
    return true;
  } else {
    console.log(
      `Reprovado por frequência. Frequência total: ${parseInt(porcentagemFrequenciaTotal)}%`,
    ); // exemplo de template literals
    return false; // reprovado: as faltas devem ser menor ou igual a 25% do total de aulas
  }
}

verificacaoAprovacaoPorFrequencia();
Reprovado por frequência. Frequência total: 68%
```

Exercício 1

❖ Terceira função:

```
function verificacaoDeAcademicoAprovado() {  
  if (  
    verificacaoAprovacaoPorNotas() &&  
    verificacaoAprovacaoPorFrequencia() === true  
  ) {  
    console.log("Aprovado");  
    return true;  
  } else {  
    console.log("Reprovado");  
    return false;  
  }  
  
  //return verificacaoAprovacaoPorNotas() && verificacaoAprovacaoPorFrequencia === true ? true : false;  
}  
verificacaoDeAcademicoAprovado();  
}
```

Reprovado por frequência. Frequência total: 23%

Reprovado

Exercício 1

- ❖ Confira o código completo desse exercício no link abaixo:
- ❖ [Exemplo](#)

Exercício 2

- ❖ Agora teste você em seu navegador os scripts vistos nessa aula.
- ❖ Acesse o **arquivo de exemplo** no link abaixo.
- ❖ Copie cada bloco de código de exemplo, uma a uma na aba console das **ferramentas de desenvolvedor** do seu navegador, e analise os resultados.
- ❖ Exemplos



univille.br